

Lab 1 Gitea Docker 与 Git LFS 实验报告

学生姓名：姜丞骏 GitHub: whatokk

一、实验目的

本次实验围绕 Docker Desktop、Gitea 私有 Git 服务和 Git LFS 展开。实验目标是在本地 Windows 环境中安装 Docker Desktop，使用 Docker Compose 部署 Gitea 服务，并验证 Gitea 是否支持 Git LFS。最后创建一个包含 1GB 以上大文件的仓库，通过 Git LFS 将大文件推送到 Gitea 仓库中。

通过本次实验，需要掌握以下内容：

1. 在 Windows 环境中安装并启动 Docker Desktop。
2. 使用 Docker Compose 部署 Gitea 服务。
3. 通过浏览器访问 Gitea 并完成初始化配置。
4. 创建 Gitea 用户和仓库。
5. 开启并验证 Gitea 的 Git LFS 支持。
6. 使用 Git LFS 管理 1GB 以上的大文件。
7. 将实验过程截图和说明整理为 Markdown 文档，并提交到 GitHub 仓库。

二、实验环境

项目	内容
操作系统	Windows 11
容器运行环境	Docker Desktop
容器编排工具	Docker Compose
Git 服务	Gitea
版本管理工具	Git
大文件管理工具	Git LFS
Gitea 访问端口	3000
Gitea SSH 端口	2222
报告格式	Markdown
提交平台	GitHub

本实验使用 Docker Desktop 作为本地容器运行环境，使用 Docker Compose 启动 Gitea 容器，并将 Gitea 数据持久化保存到本地目录中。

三、实验内容概述

本次实验主要完成以下任务：

1. 安装并检查 Docker Desktop。
2. 使用 Docker Compose 安装 Gitea。
3. 在浏览器中初始化 Gitea。
4. 验证 Gitea 是否支持 Git LFS。
5. 创建一个仓库，并推送 1GB 以上大文件。
6. 截图记录关键步骤，整理实验报告。

实验整体流程如下：



四、实验步骤与结果

4.1 安装并检查 Docker Desktop

Windows 环境下，首先需要启用 WSL 并安装 Docker Desktop。可以在 PowerShell 中执行：

```
wsl --install
```

安装完成后，下载并安装 Docker Desktop。启动 Docker Desktop 后，在 PowerShell 中检查 Docker 是否可用。

```
docker version
docker compose version
```

实验结果如下图所示。

```

C:\Users\xiaofanfan>docker version
Client:
 Version:           28.5.1
 API version:       1.51
 Go version:        gol.24.8
 Git commit:        e180ab8
 Built:             Wed Oct  8 12:19:16 2025
 OS/Arch:           windows/amd64
 Context:           desktop-linux

Server: Docker Desktop 4.48.0 (207573)
Engine:
 Version:           28.5.1
 API version:       1.51 (minimum version 1.24)
 Go version:        gol.24.8
 Git commit:        f8215cc
 Built:             Wed Oct  8 12:17:24 2025
 OS/Arch:           linux/amd64
 Experimental:      false
containerd:
 Version:           1.7.27
 GitCommit:         05044ec0a9a75232cad458027ca83437aae3f4da
runc:
 Version:           1.2.5
 GitCommit:         v1.2.5-0-g59923ef
docker-init:
 Version:           0.19.0
 GitCommit:         de40ad0

C:\Users\xiaofanfan>docker compose version
Docker Compose version v2.40.0-desktop.1

```

Docker 版本检查

从截图可以看到，Docker Client、Docker Server 和 Docker Compose 均能够正常输出版本信息，说明 Docker Desktop 已经安装成功并正常运行。

4.2 创建 Gitea 实验目录

为了便于管理实验文件，创建单独的实验目录。

```

mkdir D:\money\20260507\lab1-gitea
cd D:\money\20260507\lab1-gitea
mkdir gitea
mkdir gitea\data
mkdir gitea\config

```

目录结构如下：

```

lab1-gitea/
├── docker-compose.yml
└── gitea/
    ├── data/
    └── config/

```

其中 `gitea/data` 用于持久化保存 Gitea 的仓库、数据库和附件等数据。

4.3 编写 docker-compose.yml

在实验目录下创建 `docker-compose.yml` 文件。

notepad `docker-compose.yml`

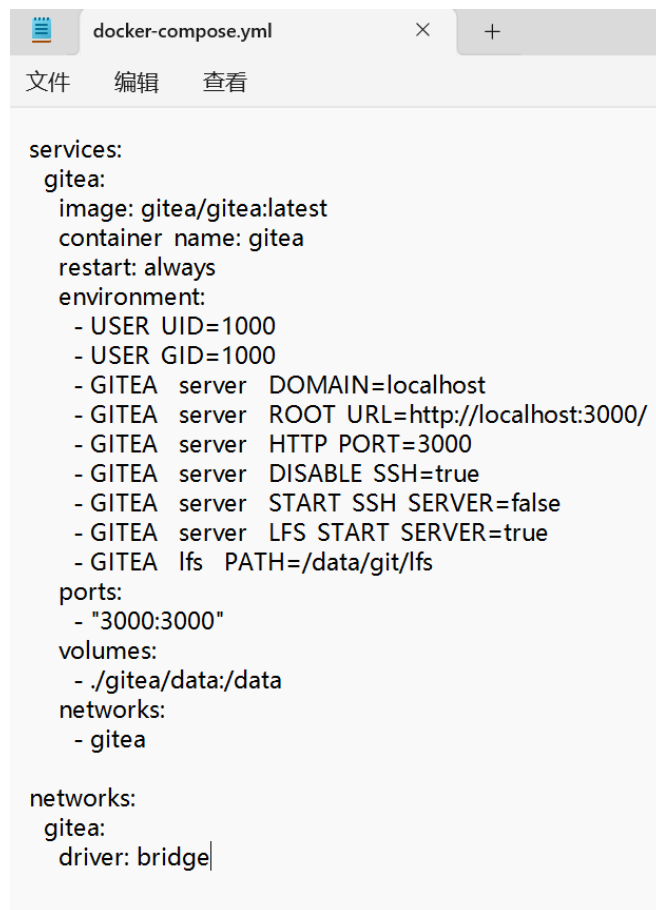
写入以下内容：

```
services:
  gitea:
    image: gitea/gitea:latest
    container_name: gitea
    restart: always
    environment:
      - USER_UID=1000
      - USER_GID=1000
      - GITEA__server__DOMAIN=localhost
      - GITEA__server__ROOT_URL=http://localhost:3000/
      - GITEA__server__HTTP_PORT=3000
      - GITEA__server__SSH_DOMAIN=localhost
      - GITEA__server__SSH_PORT=2222
      - GITEA__server__START_SSH_SERVER=true
      - GITEA__server__LFS_START_SERVER=true
      - GITEA__lfs__PATH=/data/git/lfs
    ports:
      - "3000:3000"
      - "2222:22"
    volumes:
      - ./gitea/data:/data
    networks:
      - gitea

networks:
  gitea:
    driver: bridge
```

该配置使用 `gitea/gitea:latest` 镜像启动 Gitea 服务，将容器内部 `/data` 目录映射到本地 `./gitea/data`，并开放 HTTP 端口 `3000` 和 SSH 端口 `2222`。其中 `GITEA__server__LFS_START_SERVER=true` 用于开启 Gitea 的 Git LFS 服务。

实验结果如下图所示。



```
services:
  gitea:
    image: gitea/gitea:latest
    container_name: gitea
    restart: always
    environment:
      - USER_UID=1000
      - USER_GID=1000
      - GITEA_server_DOMAIN=localhost
      - GITEA_server_ROOT_URL=http://localhost:3000/
      - GITEA_server_HTTP_PORT=3000
      - GITEA_server_DISABLE_SSH=true
      - GITEA_server_START_SSH_SERVER=false
      - GITEA_server_LFS_START_SERVER=true
      - GITEA_lfs_PATH=/data/git/lfs
    ports:
      - "3000:3000"
    volumes:
      - ./gitea/data:/data
    networks:
      - gitea

networks:
  gitea:
    driver: bridge
```

docker-compose 配置文件

4.4 启动 Gitea 容器

在 `docker-compose.yml` 所在目录执行：

```
docker compose up -d
```

查看容器运行状态：

```
docker compose ps
docker ps
```

查看 Gitea 日志：

```
docker logs gitea --tail 50
```

实验结果如下图所示。

```
D:\money\20260507\lab1>docker compose ps
NAME          IMAGE          COMMAND          SERVICE    CREATED      STATUS      PORTS
gitea         gitea/gitea:latest  "/usr/bin/entrypoint..."  gitea      54 seconds ago  Up 53 seconds  0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp, 0.0.0.0:2222->22/tcp, [::]:2222->22/tcp

D:\money\20260507\lab1>docker ps
CONTAINER ID   IMAGE          COMMAND          CREATED      STATUS      PORTS
ae8b4f19ca20   gitea/gitea:latest  "/usr/bin/entrypoint..."  55 seconds ago  Up 55 seconds  0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp, 0.0.0.0:2222->22/tcp, [::]:2222->22/tcp

D:\money\20260507\lab1>docker logs gitea --tail 50
Generating /data/ssh/ssh_host_ed25519_key...
Generating /data/ssh/ssh_host_rsa_key...
Generating /data/ssh/ssh_host_ecdsa_key...
Server listening on :: port 22.
Server listening on 0.0.0.0 port 22.
2026/05/18 03:19:03 cmd/web.go:263:runWeb() [I] Starting Gitea on PID: 15
2026/05/18 03:19:05 cmd/web.go:115:showWebStartupMessage() [I] Gitea version: 1.26.1 built with go1.26.2-X:jsonv2 : bindata, timetzdata, sqlite, sqlite_unlock_notify
2026/05/18 03:19:05 cmd/web.go:116:showWebStartupMessage() [I] * RunMode: prod
2026/05/18 03:19:05 cmd/web.go:117:showWebStartupMessage() [I] * AppPath: /usr/local/bin/gitea
2026/05/18 03:19:05 cmd/web.go:118:showWebStartupMessage() [I] * WorkPath: /data/gitea
2026/05/18 03:19:05 cmd/web.go:119:showWebStartupMessage() [I] * CustomPath: /data/gitea
2026/05/18 03:19:05 cmd/web.go:120:showWebStartupMessage() [I] * ConfigFile: /data/gitea/conf/app.ini
2026/05/18 03:19:05 cmd/web.go:121:showWebStartupMessage() [I] Prepare to run install page
2026/05/18 03:19:06 cmd/web.go:329:listen() [I] Listen: http://0.0.0.0:3000
2026/05/18 03:19:06 cmd/web.go:333:listen() [I] AppURL (ROOT_URL): http://localhost:3000/
2026/05/18 03:19:06 cmd/web.go:336:listen() [I] LFS server enabled
2026/05/18 03:19:06 modules/graceful/server.go:52:NewServer() [I] Starting new Web server: tcp:0.0.0.0:3000 on PID: 15
```

Gitea 容器运行状态

从截图可以看到，Gitea 容器已经成功启动，并且端口映射为 3000:3000 和 2222:22。

4.5 访问 Gitea 并完成初始化

在浏览器中访问：

```
http://localhost:3000
```

首次访问会进入 Gitea 初始化页面。主要配置如下：

配置项	内容
数据库类型	SQLite3
站点名称	Local Gitea
服务器域名	localhost
Gitea 基础 URL	http://localhost:3000/
HTTP 监听端口	3000

配置完成后，点击安装。安装完成后注册管理员用户并登录 Gitea。

实验结果如下图所示。



Gitea 初始化与登录

从截图可以看到，Gitea 已经可以通过浏览器访问，并且管理员用户可以正常登录。

4.6 验证 Gitea 是否开启 Git LFS

进入 Gitea 容器，查看配置文件。

```
docker exec -it gitea sh
cat /data/gitea/conf/app.ini | grep -i lfs
exit
```

也可以在 Windows 中查看配置文件：

```
notepad .\gitea\data\gitea\conf\app.ini
```

重点检查配置中是否包含：

```
[server]
LFS_START_SERVER = true
```

```
[lfs]
PATH = /data/git/lfs
```

实验结果如下图所示。

```
D:\money\20260507\lab1>docker exec -it gitea sh
/ # cat /data/gitea/conf/app.ini | grep -i lfs
LFS_START_SERVER = true
LFS_JWT_SECRET = Rc-bLBk8HXKAnnJ7r1FBENItQravxRu2CIIB9qycMgE
[lfs]
PATH = /data/git/lfs
/ # exit
```

Gitea LFS 配置验证

从截图可以看到，Gitea 配置文件中已经开启 `LFS_START_SERVER = true`，说明 Gitea 具备 Git LFS 服务端支持。

4.7 创建 Gitea 仓库

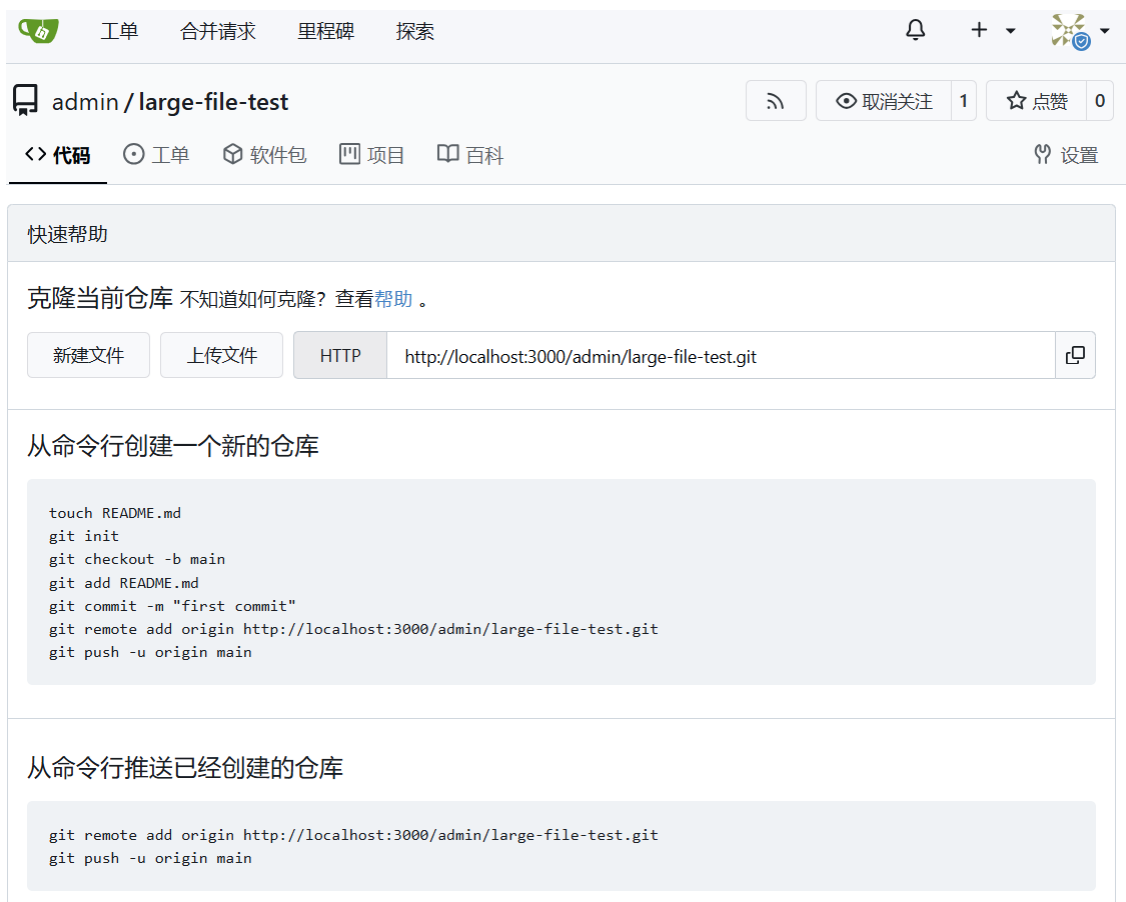
在 Gitea 页面中创建一个新仓库，例如：

large-file-test

仓库创建完成后，进入仓库页面，复制仓库 HTTP 地址。例如：

`http://localhost:3000/gitea_admin/large-file-test.git`

实验结果如下图所示。



创建 Gitea 仓库

4.8 本地初始化 Git 仓库并配置 Git LFS

在本地创建测试目录。

```
cd D:\money\20260507\lab1-gitea
mkdir large-file-test
cd large-file-test
```

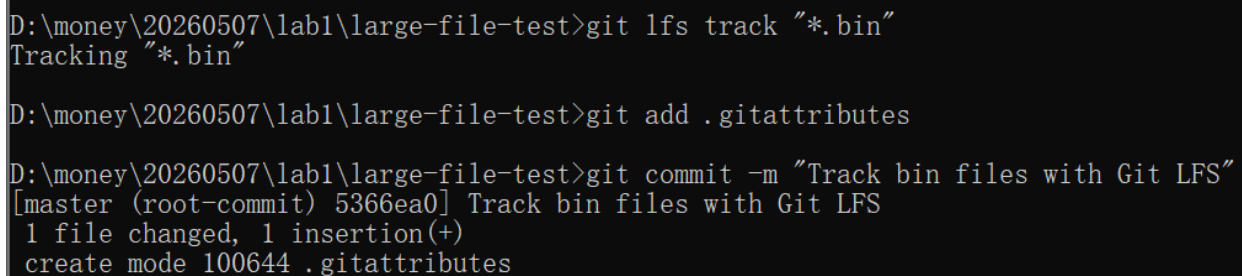
初始化 Git 仓库。

```
git init
git lfs install
```

设置 Git LFS 追踪 `.bin` 文件。

```
git lfs track &quot;*.bin&quot;
git add .gitattributes
git commit -m &quot;Track bin files with Git LFS&quot;
```

实验结果如下图所示。



```
D:\money\20260507\lab1\large-file-test>git lfs track "*.bin"
Tracking "*.bin"

D:\money\20260507\lab1\large-file-test>git add .gitattributes

D:\money\20260507\lab1\large-file-test>git commit -m "Track bin files with Git LFS"
[master (root-commit) 5366ea0] Track bin files with Git LFS
1 file changed, 1 insertion(+)
create mode 100644 .gitattributes
```

Git LFS 初始化

从截图可以看到，Git LFS 已经初始化，并且 `.bin` 文件被配置为由 Git LFS 追踪。

4.9 创建 1GB 以上大文件

在 Windows PowerShell 中可以使用以下命令创建 1GB 文件：

```
fsutil file createnew large-file.bin 1073741824
```

如果需要确认文件大小，可以执行：

```
Get-Item .\large-file.bin | Select-Object Name, Length
```

也可以使用 PowerShell 创建文件：

```
$fs = [System.IO.File]::Create(&quot;large-file.bin&quot;)
$fs.SetLength(1073741824)
$fs.Close()
```

实验结果如下图所示。

```
(base) PS D:\money\20260507\lab1\large-file-test> Get-Item .\large-file.bin | Select-Object Name,Length
Name                               Length
----                               -
large-file.bin 1073741824
```

创建 1GB 大文件

从截图可以看到，`large-file.bin` 文件大小为 1073741824 字节，即 1GB。

4.10 提交并推送大文件到 Gitea

将大文件加入 Git 仓库。

```
git add large-file.bin
git commit -m "Add 1GB large file with Git LFS"
```

添加 Gitea 远程仓库地址。

```
git remote add origin http://localhost:3000/gitea_admin/large-file-test.git
```

如果用户名不同，需要将 `gitea_admin` 替换为实际用户名。

推送到 Gitea:

```
git branch -M main
git push -u origin main
```

实验结果如下图所示。

```
(base) PS D:\money\20260507\lab1\large-file-test> git push -u origin main
Locking support detected on remote "origin". Consider enabling it with:
$ git config lfs.http://localhost:3000/admin/large-file-test.git/info/lfs.locksverify true
Uploading LFS objects: 100% (1/1), 1.1 GB | 16 MB/s, done
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 16 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (6/6), 650 bytes | 650.00 KiB/s, done.
Total 6 (delta 0), reused 0 (delta 0)
remote: . Processing 1 references
remote: Processed 1 references in total
To http://localhost:3000/admin/large-file-test.git
 * [new branch]      main -> main
Branch 'main' set up to track remote branch 'main' from 'origin'.
```

推送大文件到 Gitea

从截图可以看到，大文件已经通过 Git LFS 上传到 Gitea 仓库中。推送过程中会出现 Git LFS 上传对象的进度信息。

4.11 验证大文件由 Git LFS 管理

在本地执行：

```
git lfs ls-files
git lfs status
```

实验结果如下图所示。

```
(base) PS D:\money\20260507\lab1\large-file-test> git lfs ls-files
49bc20df15 * large-file.bin
```

Git LFS 文件验证

从截图可以看到，`large-file.bin` 已经出现在 Git LFS 管理的文件列表中。

同时在 Gitea 网页中打开仓库，可以看到 `large-file.bin` 文件已经存在于仓库中。若点击文件详情，可以看到该文件由 LFS 管理或显示为大文件对象。

实验结果如下图所示。



Gitea 页面验证 LFS 文件

4.12 查看 Gitea 数据持久化目录

Gitea 的数据被保存到本地挂载目录中。可以查看本地目录：

```
Get-ChildItem .\gitea\data
Get-ChildItem .\gitea\data\git
```

也可以查看 LFS 数据目录：

```
Get-ChildItem .\gitea\data\git\lfs -Recurse
```

实验结果如下图所示。

```
(base) PS D:\money\20260507\lab1> Get-Childitem .\gitea\data\git\lfs -Recurse

目录: D:\money\20260507\lab1\gitea\data\git\lfs

Mode                LastWriteTime         Length Name
----                -
d-----          2026/5/18      11:45             49
d-----          2026/5/18      11:46             tmp

目录: D:\money\20260507\lab1\gitea\data\git\lfs\49

Mode                LastWriteTime         Length Name
----                -
d-----          2026/5/18      11:46             bc

目录: D:\money\20260507\lab1\gitea\data\git\lfs\49\bc

Mode                LastWriteTime         Length Name
----                -
-a-----          2026/5/18      11:46 1073741824 20df15e412a64472421e13fe86ff1c5165e18b2afccf160d4dc19
                                          fe68a14

Gitea 数据持久化目录
```

从截图可以看到，Gitea 的仓库数据和 LFS 对象已经保存到本地挂载目录中。说明即使容器被删除，只要本地数据目录保留，Gitea 数据也可以继续恢复使用。

五、实验结果分析

本次实验成功使用 Docker Compose 在本地部署了 Gitea 服务。通过浏览器访问 <http://localhost:3000> 完成初始化配置后，Gitea 可以作为本地 Git 服务使用。

实验中通过环境变量 `GITEA__server__LFS_START_SERVER=true` 开启了 Gitea 内置 Git LFS 服务，并在配置文件中验证了 `LFS_START_SERVER = true`。随后在本地 Git 仓库中使用 `git lfs track "*.bin"` 配置大文件追踪，并创建了 1GB 大文件 `large-file.bin`。通过 `git push` 将该文件推送到 Gitea 仓库后，使用 `git lfs ls-files` 验证该文件已经由 Git LFS 管理。

因此可以证明：本次部署的 Gitea 支持 Git LFS，并且能够成功管理和存储 1GB 以上的大文件。

六、实验中遇到的问题与解决方法

6.1 Docker Desktop 未启动

如果执行 `docker version` 时出现无法连接 Docker daemon 的错误，通常是 Docker Desktop 没有启动。

解决方法：

打开 Docker Desktop，等待左下角显示 Docker Engine running。

然后重新执行：

```
docker version
```

6.2 Gitea 页面无法访问

如果无法访问：

```
http://localhost:3000
```

需要检查容器是否启动：

```
docker compose ps
docker logs gitea --tail 50
```

同时检查端口是否被占用：

```
netstat -ano | findstr 3000
```

如果 3000 端口被占用，可以修改 `docker-compose.yml` 中端口映射，例如：

```
ports:
  - "3001:3000"
```

此时浏览器访问：

```
http://localhost:3001
```

6.3 Git LFS 命令不可用

如果执行 `git lfs install` 时提示命令不存在，说明本地尚未安装 Git LFS。

解决方法：

```
winget install GitHub.GitLFS
```

安装完成后重新打开 PowerShell，再执行：

```
git lfs install
```

6.4 Git 推送时认证失败

如果执行 `git push` 时认证失败，需要确认 Gitea 用户名和密码是否正确。如果 Gitea 禁用了密码登录或需要 token，可以在 Gitea 页面中创建 Access Token，并使用 token 作为 Git 推送密码。

6.5 大文件没有进入 LFS

如果大文件被普通 Git 管理，通常是因为没有提前执行：

```
git lfs track "*.bin"
```

解决方法是先配置 `.gitattributes`，再重新添加文件：

```
git rm --cached large-file.bin
git add .gitattributes
git add large-file.bin
git commit -m "Move large file to Git LFS"
```

七、清理实验资源

如果实验完成后需要停止 Gitea，可以执行：

```
docker compose down
```

该命令会停止并删除容器，但不会删除本地挂载的数据目录。

如果需要完全删除 Gitea 数据，可以执行：

```
docker compose down
Remove-Item .\gitea\data -Recurse -Force
```

如果只需要重新启动 Gitea：

```
docker compose up -d
```

八、可选实验：备份与恢复

实验要求中备份与恢复为可选任务。如果需要完成，可以将 `gitea/data` 目录复制到外部硬盘或 U 盘。

备份命令示例：

```
docker compose down
Copy-Item .\gitea\data E:\gitea-backup\data -Recurse
```

恢复时，将备份目录复制回实验目录：

```
Copy-Item E:\gitea-backup\data .\gitea\data -Recurse
docker compose up -d
```

恢复后访问：

```
http://localhost:3000
```

如果仓库、用户和 LFS 文件仍然存在，说明备份恢复成功。

九、实验总结

本次实验完成了 Docker Desktop 环境检查、Gitea Docker 部署、Gitea 初始化配置、Git LFS 开启验证以及 1GB 大文件上传测试。

通过本次实验，可以得到以下结论：

1. Docker Compose 可以快速部署本地 Gitea 服务。

2. Gitea 的数据目录可以通过 volume 映射实现持久化保存。
3. Gitea 支持 Git LFS，但需要在配置中开启 `LFS_START_SERVER`。
4. Git LFS 可以将大文件内容存储为 LFS 对象，避免普通 Git 仓库直接管理大文件内容。
5. 通过 `git lfs ls-files` 和 Gitea 网页仓库，可以验证 1GB 大文件已经成功由 LFS 管理。
6. Gitea 本地部署适合用于小型团队、课程实验和私有代码仓库管理。

整体来看，本次实验完成了从 Docker 环境准备到 Gitea 私有 Git 服务部署，再到 Git LFS 大文件管理验证的完整流程。

十、参考资料

1. Docker Desktop Windows 安装文档
<https://docs.docker.com/desktop/setup/install/windows-install/>
2. Docker Desktop WSL 2 后端说明
<https://docs.docker.com/desktop/features/wsl/>
3. Gitea Docker 安装文档
<https://docs.gitea.com/installation/install-with-docker>
4. Gitea Git LFS 配置文档
<https://docs.gitea.com/administration/git-lfs-setup>